Projektowanie i analiza algorytmów	
Kierunek *Informatyczne Systemy Automatyki*	**Termin** *śr.* $17^{05} - 18^{45}$
Imię, nazwisko, numer albumu *Igor Olewicz 284126*	**Data** *22.04.2026*
Link do projektu `https://github.com/CZIWELLO/algorytmy-sortowanie`	

Politechnika Wrocławska

Sprawozdanie – projekt 1

Spis treści

1 Wstęp **1**

2 Opis algorytmów **2**
2.1 Sortowanie szybkie (quicksort) 2
2.2 Sortowanie przez scalanie (merge sort) 2
2.3 Sortowanie introspektywne (introsort) 3

3 Metodologia eksperymentów **3**

4 Wyniki **3**

5 Analiza wyników **5**

6 Wnioski **5**

7 Literatura oraz inne źródła **6**

1 Wstęp

Celem projektu była implementacja oraz analiza efektywności trzech algorytmów sortowania: sortowania szybkiego (quicksort), sortowania przez scalanie (merge sort) oraz sortowania introspektywnego (introsort).

Algorytmy te należą do najczęściej stosowanych metod sortowania danych i różnią się zarówno podejściem do problemu, jak i właściwościami czasowymi oraz pamięciowymi. W ramach projektu przeprowadzono eksperymenty mające na celu porównanie ich wydajności dla różnych typów danych wejściowych.

2 Opis algorytmów

2.1 Sortowanie szybkie (quicksort)

Sortowanie szybkie jest algorytmem opartym na strategii „dziel i zwyciężaj”. W niniejszym projekcie wykorzystano wariant oparty na schemacie Hoare'a, w którym podział tablicy odbywa się przy użyciu dwóch wskaźników przemieszczających się od początku i końca tablicy.

Kluczowym elementem algorytmu jest wybór elementu osiowego (pivotu), względem którego dzielona jest tablica. W implementacji zastosowano strategię wyboru pivota jako mediany z trzech elementów (pierwszego, środkowego oraz ostatniego). Takie podejście pozwala ograniczyć ryzyko wystąpienia skrajnie nierównomiernych podziałów, które negatywnie wpływają na wydajność algorytmu.

Podczas działania algorytmu tablica dzielona jest na dwie części:

- elementy mniejsze od pivota,
- elementy większe od pivota.

Proces podziału realizowany jest poprzez przesuwanie dwóch wskaźników: jednego od lewej strony tablicy oraz drugiego od prawej. W momencie znalezienia elementów znajdujących się po niewłaściwej stronie pivota następuje ich zamiana. Po zakończeniu tego etapu otrzymujemy granicę podziału, która wyznacza dwie podtablice.

Algorytm następnie rekurencyjnie sortuje obie części. W celu ograniczenia zużycia pamięci zastosowano optymalizację polegającą na rekurencyjnym sortowaniu mniejszej części tablicy, natomiast większa część przetwarzana jest iteracyjnie.

Złożoność obliczeniowa:

- przypadek średni: $O(n \log n)$
- przypadek najgorszy: $O(n^2)$

Zastosowanie mediany z trzech elementów jako pivota znacząco zmniejsza prawdopodobieństwo wystąpienia najgorszego przypadku, poprawiając stabilność działania algorytmu w praktyce.

2.2 Sortowanie przez scalanie (merge sort)

Sortowanie przez scalanie również wykorzystuje strategię „dziel i zwyciężaj”. Algorytm polega na rekurencyjnym dzieleniu tablicy na dwie części aż do uzyskania podtablic jednoelementowych.

Następnie podtablice te są scalane w taki sposób, aby uzyskać większe posortowane fragmenty, aż do uzyskania jednej, w pełni posortowanej tablicy.

Kluczowym etapem algorytmu jest operacja scalania dwóch posortowanych podtablic w jedną.

Złożoność obliczeniowa:

- przypadek średni: $O(n \log n)$
- przypadek najgorszy: $O(n \log n)$

Algorytm wymaga dodatkowej pamięci rzędu $O(n)$ na potrzeby scalania.

2.3 Sortowanie introspektywne (introsort)

Sortowanie introspektywne jest algorytmem hybrydowym, łączącym zalety sortowania szybkiego oraz sortowania przez kopcowanie.

Algorytm rozpoczyna działanie jako quicksort. Monitorowana jest jednak głębokość rekurencji. Jeśli przekroczy ona ustalony limit, algorytm przełącza się na sortowanie przez kopcowanie, które gwarantuje złożoność $O(n \log n)$ w najgorszym przypadku.

Złożoność obliczeniowa:

- przypadek średni: $O(n \log n)$
- przypadek najgorszy: $O(n \log n)$

Dzięki połączeniu kilku metod introsort łączy wysoką wydajność quicksorta z bezpieczeństwem heapsorta.

3 Metodologia eksperymentów

Eksperymenty przeprowadzono dla tablic o rozmiarach:

- 10 000,
- 50 000,
- 100 000,
- 500 000,
- 1 000 000.

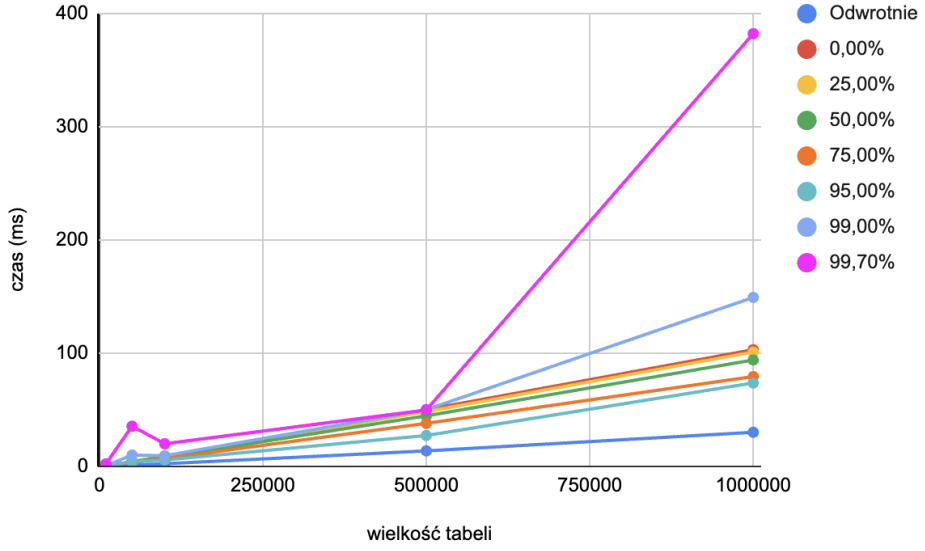
Dla każdego rozmiaru wykonano testy dla następujących przypadków:

- dane losowe,
- częściowo posortowane (25%, 50%, 75%, 95%, 99%, 99.7%),
- dane posortowane w odwrotnej kolejności.

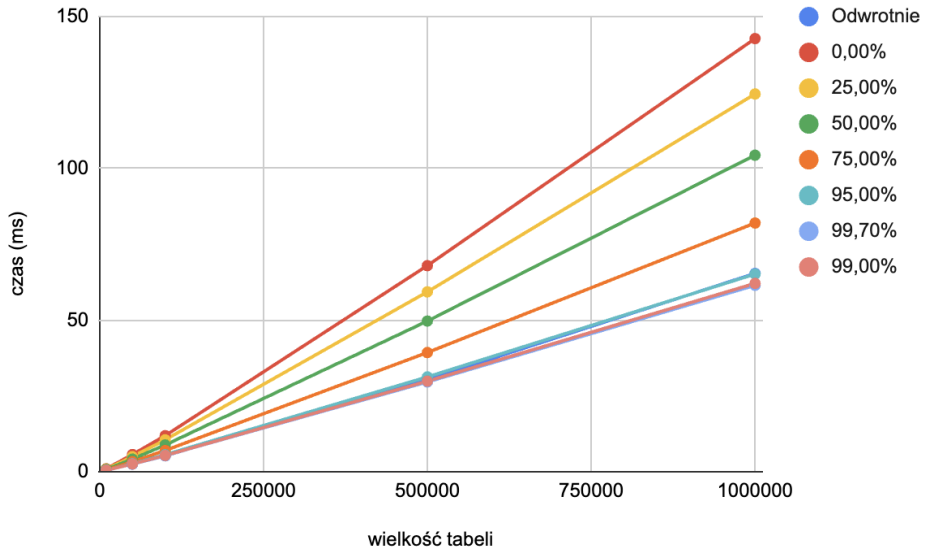
Każdy test został powtórzony stukrotnie, a wyniki uśredniono.

4 Wyniki

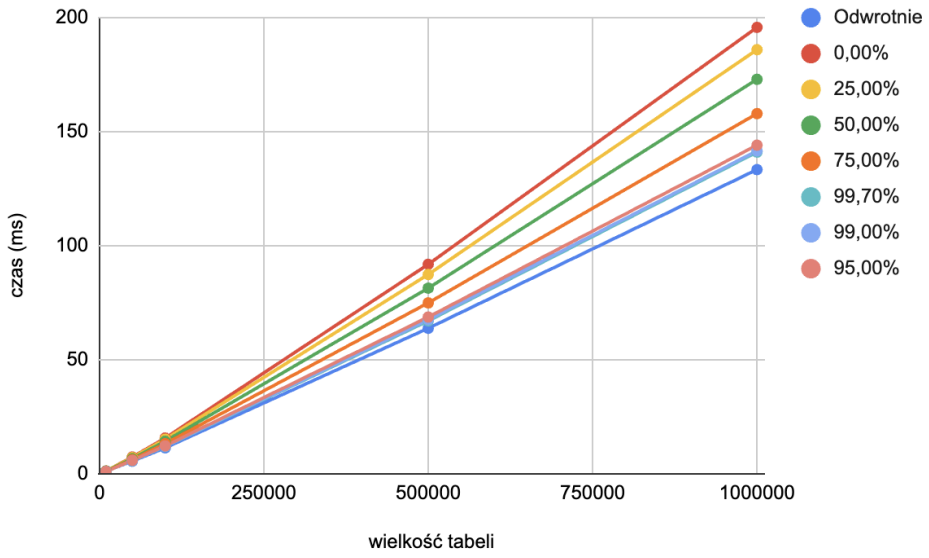
Poniżej przedstawiono przykładowe wykresy obrazujące czas działania algorytmów.



Rysunek 1: Czas działania sortowania szybkiego (ms)



Rysunek 2: Czas działania sortowania przez scalanie (ms)



Rysunek 3: Czas działania sortowania introspektywnego (ms)

5 Analiza wyników

Na podstawie przeprowadzonych eksperymentów można zauważyć, że:

- teoretycznie sortowanie szybkie osiąga bardzo dobre wyniki dla danych losowych, jednak jego wydajność pogarsza się dla danych częściowo uporządkowanych. W naszej implementacji mimo wykonania testu wiele razy oraz weryfikacji poprawności algorytmu wyniki nie pokrywają się poprawnie z teorią szczególnie dla większych rozmiarów tablic oraz poziomów posortowania. Najprawdopodobniej wynika to z problemów sprzętowych związanych z małą ilością wolnej pamięci na komputerze.
- sortowanie przez scalanie charakteryzuje się stabilną wydajnością niezależnie od rodzaju danych wejściowych,
- sortowanie introspektywne łączy zalety quicksorta i heapsorta, zapewniając zarówno wysoką wydajność, jak i odporność na niekorzystne przypadki. Mimo tego, że ten algorytm korzysta takę z funkcji sortowania quicksort, podejście hybrydowe sprawia że nie występuje problem ze zwiększeniem czasu sortowania bądź jest on ledwo zauważalny.

6 Wnioski

Analiza wykazała, że:

- quicksort jest bardzo szybki w praktyce, ale wrażliwy na niekorzystne dane wejściowe,
- merge sort zapewnia stabilną i przewidywalną wydajność kosztem dodatkowej pamięci,
- introsort stanowi najlepszy kompromis pomiędzy wydajnością a bezpieczeństwem, dlatego jest wykorzystywany w standardowych bibliotekach języka C++.

7 Literatura oraz inne źródła

- A. Drozdek, *C++. Algorytmy i struktury danych*
- wiedza o introsort: `https://www.geeksforgeeks.org/dsa/introsort-cs-sorting-weapon/`
- materiał o sortowaniu przez scalanie: `https://www.youtube.com/watch?v=3j0SWDX4AtU&t=123s`
- materiał o quicksort: `https://www.youtube.com/watch?v=Vtckgz38QHs&t=639s`